Mainline Linux on a Samsung Galaxy phone

Field guide from the Galaxy S22 (SM-S901E, Snapdragon 8 Gen 1) postmarketOS port - October 2026

Read this first if you have a Galaxy S10+. The S10+ is a very different phone underneath. Australian/European models are **SM-G975F (Samsung Exynos 9820)**; US models are **SM-G975U (Snapdragon 855)**, and US Snapdragon Samsung bootloaders generally cannot be unlocked. Check the model number in Settings > About phone before anything else. Almost every driver fix in this guide is specific to the S22's Qualcomm SM8450 chip and will NOT carry over. What carries over is the **method**: the safety rules, the flash loop, how to read the stock firmware, and how to find a reference port. Part F is the starting checklist for an S10+.

Part A - Start to finish: unlock, root, EFS backup, TWRP

Do these steps **in this order** and do not start any Linux work until step A5 is done. A boot that hangs with no logs and no recovery path costs hours; with TWRP in place every mistake is one reboot away from fixed.

A0. Install everything first (one time, needs sudo once)

Install all of this **before** touching the phone, so nothing has to be installed mid-flash. Tested on Ubuntu 24.04 / Pop!_OS. Log out and back in afterwards (plugdev group).

```
sudo apt-get update
sudo apt-get install -y adb android-sdk-platform-tools-common udev git rsync make \
  build-essential bc flex bison libssl-dev libelf-dev libncurses-dev openssl \
  gcc-aarch64-linux-gnu device-tree-compiler zstd lz4 cpio python3 python3-venv \
  python3-tk pipx libusb-1.0-0 network-manager e2fsprogs patch curl kmod pciutils usbmuxd
sudo usermod -aG plugdev "$USER"
pipx install pmbootstrap && pipx ensurepath          # no sudo
# Odin4-GUI (Samsung flashing on Linux) in its own venv, no sudo to install:
git clone https://github.com/Ragekill3377/Odin4-GUI && cd Odin4-GUI/Odin4-GUI
sudo ./install_odin4.sh                               # installs the odin4 binary (one time)
python3 -m venv venv && venv/bin/pip install -r requirements.txt   # customtkinter, no sudo
sudo venv/bin/python main.py                           # run (needs USB access)
# before EVERY flash: ModemManager grabs the phone's USB port
sudo systemctl stop ModemManager
```

• Why each matters: `adb` (TWRP/flash loop), `lz4` (Samsung .lz4 images), `e2fsprogs` (debugfs: read MAC addresses from the EFS backup), `gcc-aarch64-linux-gnu, flex, bison, libssl-dev, libelf-dev, bc` (kernel build), `device-tree-compiler` (dtc), `zstd` (compressed modules), `python3-tk` (Odin4-GUI window), `pmbootstrap` (builds the postmarketOS image).

• Downloads to have ready: stock firmware for the **exact** model, region and build already on the phone (S22: SM-S901E XSA S901EXXSCFYH1; from samfw.com or SamMobile: BL_, AP_, CP_, CSC_, HOME_CSC_ .tar.md5), TWRP for the device codename (S22: r0q; S10+: beyond2lte), the Magisk APK (github.com/topjohnwu/Magisk), and Samsung's kernel source for the model (opensource.samsung.com).

• Keep kernel/source folders in a path **without spaces** (e.g. ~/s22-kernel) - the kernel build refuses spaces.

Phone-side packages: add them when building the image so you never install over a slow link mid-debug:

```
pmbootstrap init        # device samsung-r0q, UI plasma-mobile, init openrc
pmbootstrap install --add bluez,bluez-btmgmt,linux-firmware-ath11k,linux-firmware-qca,\
  i2c-tools,mesa-demos,chrony,iw,wireless-regdb,rmtfs,tqftpserv,qrtr,qmi-utils,\
  plasma-keyboard,nftables
```

(On the S22 deviceinfo we also set `deviceinfo_rootfs_image_sector_size="4096"` - UFS uses 4K sectors - and a fixed USB MAC so the PC-side network interface name never changes.)

A1. Unlock the bootloader (wipes the phone)

• Settings > About phone > Software information: tap Build number 7x. Developer options > **OEM unlocking** on.

• Power off. Hold **Volume Up + Volume Down** and plug in USB: the download-mode warning screen appears. **Long-press Volume Up** to unlock, confirm. The phone wipes and reboots.

• Set the phone up again (skip accounts), enable Developer options again and check OEM unlocking is on and greyed out. Knox trips permanently (Samsung Pay/Secure Folder stop working) - that is expected.

A2. Root with Magisk (so you can read the EFS partitions)

• Copy the **AP_....tar.md5** to the phone, install the Magisk app, Magisk > Install > Select and patch a file > the AP tar. Copy the resulting `magisk_patched-xxxxx.tar` back to the PC.

• Download mode again (Vol Up + Vol Down + USB). In Odin: **BL** = BL_, **AP** = magisk_patched tar, **CP** = CP_, **CSC** = **HOME_CSC_** (HOME_ keeps data; plain CSC_ wipes). Start.

• Boot, open Magisk and let it finish setup. Check root: `adb shell su -c id` must show uid=0.

A3. Back up EFS and friends - before anything else

These partitions hold the IMEI, Wi-Fi/Bluetooth MAC addresses and calibration. They are unique to the phone and cannot be recreated. Back them up as raw images:

```
adb shell
su
cd /dev/block/by-name
for p in efs sec_efs persist modemst1 modemst2 fsg fsc; do
  dd if=$p of=/sdcard/$p.img
done
exit; exit
mkdir efs-backup && cd efs-backup
for p in efs sec_efs persist modemst1 modemst2 fsg fsc; do adb pull /sdcard/$p.img; done
sha256sum *.img > SHA256SUMS      # keep this file
adb shell 'rm /sdcard/*.img'
```

• Copy the folder somewhere off the PC too (USB stick / cloud). Check later with `sha256sum -c SHA256SUMS`.

• Partition names differ per device - list them first: `ls -l /dev/block/by-name`. Never write to these partitions again; on the S22 the modem now uses *file copies* of modemst1/2, fsg, fsc.

• Useful later: the Bluetooth address is in efs /bluetooth/bt_addr, the Wi-Fi MAC in efs /wifi/.mac.info (read from the image on the PC with `debugfs -R 'cat /wifi/.mac.info' efs.img`).

A4. Install TWRP to the recovery partition - straight away

• Download mode. Odin: **AP** = the TWRP .tar, **untick Auto Reboot**, Start.

• When it says PASS, hold **Volume Down + Power** until the screen goes off, then immediately **Volume Up + Power** (with USB in) to boot straight into TWRP. If stock Android boots first it can replace TWRP with stock recovery.

• Follow the device's own TWRP page for extras (some Samsungs, including the S10 family, need vbmeta / a 'multidisabler' zip). TWRP stays in recovery for good: from now on you only ever flash **boot**.

A5. The software-only flash loop

• Linux -> TWRP: a tiny helper `reboot2.c` calling `reboot(LINUX_REBOOT_CMD_RESTART2, "recovery")` (busybox 'reboot recovery' ignores the argument). Always pass an argument.

• TWRP -> Linux: `adb reboot`. If Linux hangs: hold Power + Volume Down ~10 s.

• Linux cannot be read from TWRP's old kernel (ext4 orphan_file) - read logs over SSH or from pstore instead.

```
ssh phone 'sync; sudo ~/reboot2 recovery'
adb push boot.img /tmp/ && adb shell 'sha256sum /tmp/boot.img'          # must match the PC
adb shell 'readlink -f /dev/block/by-name/boot'                         # check before writing
adb shell 'dd if=/tmp/boot.img of=/dev/block/by-name/boot bs=4096 && sync'
adb shell 'dd if=/dev/block/by-name/boot bs=4096 count=24576 | sha256sum'   # read-back must match
adb reboot
```

A6. Do USB-C auto-detection as early as possible. Without it the phone's USB network only comes up after a cable replug on every boot (no attach event reaches the USB controller). On Samsung phones with a MAX77705 companion chip the fix is the MAX77705 Type-C (CCIC) driver plus a USB role switch in the DT (dr_mode "otg", role-switch-default-mode "peripheral", connector endpoint linked to the USB controller). On the S22 we only did this at attempt 23 - doing it right after USB networking first works would have saved a replug on every single test boot. Patches: kernel-config/upstream-patches/tabs8/.

Part B - Linux bring-up, in the order it happened

Step	What we did	Result
1-3	uniLoader in boot slot; DTB with explicit RAM ranges + Samsung no-map carve-outs; /dev/console in initramfs	Kernel runs, initramfs shell
4-5	USB gadget networking; reboot2 helper	Root shell over USB; button-free reboot to TWRP
6-8	UFS + clock controllers; pmOS rootfs rebuilt with 4K sectors	postmarketOS boots from internal storage, SSH
10-11	Modules one at a time (qcrypto hangs the SoC - keep blocked); touch driver; antenna-switch rail	Bluetooth, touch, battery
12	Adreno 730 with Samsung-signed zap shader from the phone	GPU works
13	New S6E3FAC panel driver; Samsung source showed missing DCS 0x53	Real display
15-16	Fuel gauge / charger DT links	Battery %, charging
17	Found mainline bug: pcie-qcom parsed iommu-map with the wrong entry size	Wi-Fi interrupts arrive, firmware loads
19	Matched the Galaxy Tab S8+ port's Wi-Fi DT (rails, no IOVA pool, mainline PHY, upstream firmware)	Wi-Fi up and scanning
20	Modem: Samsung modem firmware + rmtfs serving EFS copies read-only	Modem boots, QMI online
21	Removed mte_mem (512 MiB) that stock never reserves	+512 MiB RAM
-	Panel module: stock fhd120hs timing + 3 command bytes	120 Hz
22	nftables in kernel; hwclock service removed	Firewall up, clean boot
23	MAX77705 Type-C driver (Tab S8+ patches), USB dual-role	USB connects without replug

Part C - What we found (problems and their causes)

Symptom	Cause / fix
Hang right after 'Booting Kernel'	DTB memory node had size 0 - give explicit RAM ranges
init panic, no console	No /dev/console in initramfs
pmOS cannot find its rootfs	UFS uses 4096-byte sectors - build the image with 4K sectors
Phone freezes when modules auto-load	qcrypto (crypto engine) is TrustZone-owned - blocklist it forever
Bluetooth finds nothing	Antenna-switch rail (PMR735A L7, 2.8 V) must be always-on
Display black with correct init	Missing DCS 0x53 (brightness control) - found in Samsung's panel source
Wi-Fi: firmware download 'fails'	Interrupts lost: mainline pcie-qcom misread iommu-map (5-cell entries) - upstreamable patch written
Wi-Fi: chip bootloader crashes	Fixed by copying the Galaxy Tab S8+ DT (same SoC, same Wi-Fi chip)
Wi-Fi: cannot connect	OPEN: signal ~25 dB weak, router never hears the phone - antenna/RF path
Only 6.2 of 8 GB RAM	Mainline reference reservations (mte_mem) - delete what stock does not reserve
USB needs a replug every boot	No attach event - MAX77705 Type-C driver + role switch
'Kernel lacks nftables' at boot	Enable NF_TABLES (changes struct net: rebuild image AND all modules)

Part D - What works today

Works	Partly	Not yet
Boot from internal storage, USB SSH (no replug), display 120 Hz, touch, GPU, Bluetooth, charging, battery %, 6.6 GiB RAM, firewall, on-screen keyboard, Plasma Mobile	Wi-Fi (scans, can't connect), modem (boots, no SIM test), USB host/OTG (untested)	Audio (in progress), calls/SMS, mobile data, sensors, camera, fingerprint, NFC, suspend

Part E - Lessons that transfer to any Samsung phone

• Read the phone's **stock device tree** and **Samsung's kernel source for the exact model** before guessing.

• Find a **working port on the same SoC** and diff against it - for the S22 the Galaxy Tab S8+ port solved Wi-Fi, USB-C and audio wiring.

• Use the phone's **own firmware files** (GPU zap, ADSP, modem) from its firmware partitions.

• One change per test; log everything; re-test 'ruled out' ideas after fixing something more basic.

• Get SSH over USB early - most work is then live, without flashing - and add USB-C attach detection right after it (no more replugging).

Part F - Starting checklist for a Galaxy S10+

• Model: SM-G975F (Exynos 9820) is the unlockable one in AU/EU. SM-G975U (Snapdragon, US) is usually locked - stop there.

• Do Part A exactly (unlock, Magisk root, EFS backup, TWRP for beyond2lte) before any kernel work.

• Check postmarketOS / the Nura wiki for **samsung-beyond2lte** and any Exynos 9820 mainline work; reuse what exists. Exynos mainline support is far less complete than Qualcomm's - expect a longer road.

• Get Samsung's SM-G975F kernel source and the stock firmware; dump the stock device tree.

• uniLoader started life on Exynos phones - check whether it already has an Exynos 9820 board.

• First target: a USB shell, then USB-C attach detection ASAP (check which Type-C/PMIC chip the S10+ uses in its stock DT). Then add drivers one at a time.

Files in the S22 project folder: notes/boot-log.md (every attempt), notes/CHANGELOG.md, notes/CHECKLIST-not-sorted.md, notes/SETUP.md (host setup), notes/flashing-workflow.md, kernel-config/ (DTS, patches, config).